



유전 알고리즘을 활용한

화재 대피 우선순위 관제 시스템

남궁다정 | 김민서 | 곽민성

주제 선정 계기 보충



출처 : 연합뉴스

(<https://www.yna.co.kr/view/AKR20190924102500065>)

거동이 불편한 입원 환자들은 불이 난 것을 보면서도 대피할 방법이 없어 화재 속 공포에 떨어야 했다.

⋮

최근 무릎 고관절 수술을 받은 뒤 입원 중이던 지동심(79·여)씨는 구조대원들의 도움을 받아 가까스로 대피했다.

출처 : 연합뉴스(<https://www.yna.co.kr/view/AKR20190924102500065>)

국립재난안전연구원이 실시한 조사에 따르면 요양병원 종사자들은 화재 발생 시 가장 큰 어려움으로 '입소자의 자력 대피 곤란'과 '대피를 도울 인력 부족'을 주요 요인으로 꼽았다.

출처 : 한국재난뉴스(<https://www.hjnews.co.kr>)

주제 선정 계기

요양병원 사고는 일어날 때마다 피해가 일반 병원보다 심각



제한된 인력으로 모든 환자 동시 구조 어려움



이 상황에서 가장 필요한 게 뭘까



대피를 더 효율적으로 빠르게 할 수 있는 방법이 필요

**화재 발생 시 대피 우선순위를 알려주는
프로그램을 만들자!**

프로그램 시연 및 설명

🔥 화재층

2층

Generation

-

Penalty

-

🏠 건물 설정

화재 발생 층

2층

+ 층 추가

- 층 제거

최대 10층까지 추가 가능

🎲 랜덤 생성

🚀 AI 분석 시작

👤 환자 편집 — 선택 없음

방 카드를 클릭하면 입력창이 활성화됩니다.

💾 현재 방 데이터 저장

📋 병동 현황

1층

101호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

102호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

103호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

104호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

105호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

106호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

2층

201호

위험

👤 이상 3

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

202호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

203호

주의

👤 이상 0

🔒 밀폐여 2

🚒 치매 0

🔥 자립 0

204호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

205호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

206호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

3층

301호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

302호

매우 위험

👤 이상 4

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

303호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

304호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

305호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

306호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

4층

401호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

402호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

403호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

404호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

405호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

406호

안전

👤 이상 0

🔒 밀폐여 0

🚒 치매 0

🔥 자립 0

● 매우 위험

● 위험

● 주의

● 안전

⚡ AI 추천 대피 순위 TOP 3

AI 분석 시작 버튼을 누르면 결과가 표시됩니다.

AI 판단의 이론적 근거 ① 환자 유형 별 가중치

환자 유형(프로그램)	참고 기준 (ADL)	대피에 필요한 시간
와상 환자	ADL 매우 낮음	180초
치매 환자	피난약자 분류	90초
휠체어 환자	ADL 중간	60초
자립 환자	ADL 높음	20초

보건복지부 요양병원 환자분류의 **ADL(일상생활수행능력)** 개념과
병원 재난 대응 시 **피난약자 분류**를 참고하여, 프로그램 목적에 맞게 환자 유형을 단순화함

*ADL(Activity of Daily Living):
식사, 배설, 목욕, 옷 갈아입기, 이동 등 일상생활 수행 능력을 평가하는 지표

AI 판단의 이론적 근거

② 현재 속한 층별 가중치

층 위치	Penalty 적용	이유
화재층	대기시간 × 2.0	화염과 연기에 직접 노출되어 가장 위험
직상층	대기시간 × 1.5	연돌효과로 연기가 빠르게 확산
그 외 층	대기시간 × 1.0	기본 페널티 적용

※ 화재층과 직상층의 구조 지연에 더 큰 불이익을 부여하여
우선 구조되도록 설계

유전 알고리즘이란

실제 생물 진화를 모방해서 문제를 해결하는 알고리즘

수많은 경우의 수 안에서 '**최선**'을 찾기

→ 후보 해를 먼저 쓴 다음 평가하고, 좋은 해만 남기는 구조

유전 알고리즘의 과정

① **초기화** (Initialize): 초기 염색체 집합 무작위 생성



② **평가** (Evaluation): 초기 염색체에서 적합도 계산



③ **선택** (Selection): 계산된 적합도를 기준으로 우수한 개체(부모)들을 선별



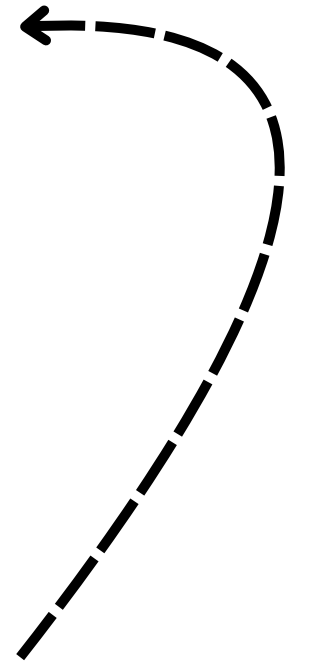
④ **교차** (Crossover): 선별된 개체(부모) 유전자를 교환 → 새로운 자식 생성



⑤ **변이** (Mutation): 자식이 생성될 때 유전자를 확률적으로 변이(돌연변이)



⑥ **반복** (Loop): ③~⑤번을 반복하며 가장 좋은 유전자를 선별



유전 알고리즘을 선택한 이유

	유전 알고리즘	완전탐색 알고리즘	탐욕 알고리즘
특징	여러 해를 반복적으로 개선하며 최적해 탐색	모든 경우를 계산하여 최적해 탐색	현재 시점의 최적화를 반복하여 최적해 탐색
장점	복잡한 문제에서도 효율적으로 좋은 해 탐색	항상 최적해 보장	계산 속도가 빠름
단점	최적해를 100% 보장하지는 않음	경우의 수가 많아질수록 계산량이 매우 큼	전체 최적해를 보장하지 못함

사용 AI / 개발 언어



Claude

코드 작성

및 디버깅 보조

프롬프트 1: 대피 우선순위를 찾는 프로그램을 개발할 예정 → 어떤 방법으로 개발?

AI 답변 1: 유전 알고리즘을 이용한 대피 우선순위를 찾는 프로그램을 추천

프롬프트 2: 유전 알고리즘을 활용한 대피 우선순위 찾기 프로그램 개발 방법?

AI 답변 2: 유전 알고리즘을 활용해 유전 알고리즘의 흐름에 맞는 코드 작성



Javascript / CSS / html

- 유전 알고리즘 로직 구현
- UI
- 화면 구성

코드 설명 - 1. 초기 염색체 무작위 생성

```
const popSize = 30; // 한 세대에 존재할 염색체(가중치 조합) 개체 수를 30으로 설정
for (let i = 0; i < popSize; i++) population.push(randomChromosome());
// randomChromosome()을 30번 호출해서 무작위 초기 개체군(1세대)을 생성
```

↑ randomChromosome 함수를 30번 반복 호출한다. 호출 후 반환값을 **population**에 쌓는다.

```
function randomChromosome() { // 무작위
  return {
    fireFloorW: getRandom(0, 120),
    adjFloorW: getRandom(0, 120),
    upperFloorW: getRandom(0, 120),
    lowerFloorW: getRandom(0, 120),
    wA: getRandom(0, 60),
    wB: getRandom(0, 60),
    wD: getRandom(0, 60),
    wC: getRandom(0, 60)
  };
}
```

← 염색체 가중치 무작위 설정

- 화재층, 직상층, 기타상층, 하부층의
가중치 0~120사이 무작위 설정

- 와상 환자, 휠체어 환자, 치매 환자, 자립 환자
가중치 0~60사이 무작위 설정

ex) population = [] 에

population[0] = fireFloorW:114.2, adjFloorW : 85.6 ...

population[1] = fireFloorW:92.5, adjFloorW : 71.3 ...

population[2] = fireFloorW:34.1, adjFloorW : 98.7 ...

:

population[29] = fireFloorW:118.7, adjFloorW: 20.2...

코드 설명 - 2. 염색체 평가

`function evaluatePenalty(chromosome) {` ← 염색체 평가 함수(패널티 부여 함수)

```
let totalPenalty = 0; // 전체 패널티 총합을 누적할 변수, 0으로 초기화
candidateFireFloors.forEach(hypoFireFloor => { // 후보로 뽑힌 각 층
  let rooms = []; // 이번 가상 화재 시나리오에서 사용할 방 목록(점수)
  for (let key in hospitalData) { // 모든 방을 다시 순회하며 이번 층의 방
    let d = hospitalData[key]; // 방의 인원 데이터
    let total = d.A + d.B + d.C + d.D; // 방의 총 인원수
    if (total === 0) continue; // 사람이 없는 빈 방은 계산할 필요 없음
    let floor = parseInt(key.split('층')[0]); // 이 방이 있는 층 번호
    let score = getFloorWeight(floor, hypoFireFloor, chromosome)
      + getPatientWeight(d, chromosome); // getFloorWeight() 호출
    rooms.push({ floor, data: d, score }); // 계산된 층/데이터/점수
  }
  rooms.sort((a, b) => b.score - a.score); // 점수가 높은(위험한) 방이 앞에 오도록 내림
  // 별점 매기기
  let elapsed = 0; // 대피 시작부터 지금까지 누적된 경과 시간(초)을 담을 변수, 0으로 초기화
  rooms.forEach(r => { // 정렬된 순서대로(위험한 방부터) 방을 하나씩 처리
    elapsed += estimateAttendSeconds(r.data); // estimateAttendSeconds() 호출 → 이 방의 대피
    if (r.floor === hypoFireFloor) totalPenalty += elapsed * 2; // 만약 이 방이 실제 화재 층이면
    else if (Math.abs(r.floor - hypoFireFloor) === 1) totalPenalty += elapsed * 1.5; // 화재 층과 인접한 층이면
    else totalPenalty += elapsed; // 그 외 층이면 가중치 없이(1배) 경과시간을 그대로 패널티 부여한다.
  });
}
```

← 각 층의 모든 방을 순회하면서 빈방을 제외한다.

사람이 있는 방이라면 그 방의 위험도를 계산한다.

score = 층에 따른 위험도 + 환자 유형에 따른 위험도

rooms = [] 배열에 [현재 층 / 환자 데이터 / score]를 세트
로 추가

rooms[0] = floor:3, data:{A:2, B:0, C:0, D:1}, score:184.4

rooms[1] = floor:1, data:{A:0, B:0, C:1, D:4}, score:64.2

⋮

← score가 높은 순으로 정렬

- 환자 유형에 따라서 대피 시간 계산
- 발화 층을 기준으로 방마다

화재층, 직상층, 하부층, 기타상층에 따라 경과 시간을 계산하고 패널티를 부여한다.

코드 설명 - 3. 선택, 교차, 변이 (반복)

```
population.forEach(c => c.penalty = evaluatePenalty(c));  
population.sort((a, b) => a.penalty - b.penalty); // 패널티  
let parents = population.slice(0, popSize / 2); // 정렬된
```

↑ **선택** : 우수한 개체 선별

- population에 패널티 값 추가

ex) population[4] = fireFloorW:114.2... +penalty:225.2

- 패널티가 낮은 순으로 정렬시키고 그중 상위 50%를 부모
염색체로 선택한다.

교차 : 부모 염색체 중에서 2개를 선택해 유전자를 섞는다→.

```
function crossover(a, b) { // 부모 염색체 두 개(a, b)를  
  const pick = (x, y) => Math.random() > 0.5 ? x : y;  
  return { // 각 유전자(가중치 항목)마다 pick()을 호출해  
    fireFloorW: pick(a.fireFloorW, b.fireFloorW), // 화재  
    adjFloorW: pick(a.adjFloorW, b.adjFloorW), // 직상  
    upperFloorW: pick(a.upperFloorW, b.upperFloorW), // 상  
    lowerFloorW: pick(a.lowerFloorW, b.lowerFloorW), // 하  
    wA: pick(a.wA, b.wA), // 와상 가중치를 부모 중 하나  
    wB: pick(a.wB, b.wB), // 휠체어 가중치를 부모 중 하나  
    wD: pick(a.wD, b.wD), // 치매 가중치를 부모 중 하나  
    wC: pick(a.wC, b.wC) // 자립 가중치를 부모 중 하나  
}
```

```
function mutate(c) { // 염색체(c)에 무작위 돌연변이를 일으키는 함수. crossover  
  const keys = ["fireFloorW", "adjFloorW", "upperFloorW", "lowerFloorW",  
    "wA", "wB", "wD", "wC"]; // 돌연변이를 적용할 수 있는 모든 유전자(가중치)  
  keys.forEach(k => { // 각 유전자 항목에 대해 반복  
    if (Math.random() < 0.1) c[k] =  
      k.startsWith("w") ? getRandom(0, 60) : getRandom(0, 120);  
    // 10% 확률로 돌연변이 발생: 항목 이름이 "w"로 시작하면(환자 유형 가중치)  
  });
```

← **변이** : 교차된 염색체 중에서 10% 확률
로 돌연변이를 발생 시켜, 새로운 가중치를
주입한다.

이유 : 단순 교배만으로는 발생할 수 있는 고
정관념 정체 현상을 방지하고, 무작위 변이
를 통해 우연히 더 우수한 가중치 조합을 발
견할 확률적 가능성을 열어두기 위함.

지속 발전 가능성

IoT 센서 연동

연기 · 온도 센서와 연동
↓
화재 정보 자동수집

화재 확산 반영

시간에 따라
화재 및 연기가
어떻게 퍼지는지
실시간으로 표시

구조 인원 고려

구조 인력 규모 반영
↓
더 현실적인 대피 계획

구조대원과 연동

대피 정보를 구조대원
에게도 공유
↓
구조 속도 극대화

감사합니다.